[PROJECT NAME TBD] — Generative Agent Town Sim

Design & Build Plan (v2 — expanded)

Builds directly on the Hollowmere prototype: Stanford "Generative Agents" (Smallville) architecture, pixel art world, live LLM-driven NPCs. This doc locks the architecture so Claude Code can build against it without re-litigating decisions mid-build.

---

1. Concept

A persistent pixel-art town where NPCs have their own daily routines, memories, relationships, and free will. They wake up, go to work, run errands, gossip, form opinions of each other and of the player, and remember what happened yesterday. The player can walk around, talk to anyone, and watch the town evolve on its own even when unobserved (simulated in the background).

Design pillars (keep these in view whenever a build decision is ambiguous):

- **Legibility over scale.** A small town where every NPC's behavior is explainable beats a big one that reads as noise. Cast size and map size stay small in v1 on purpose.
- **Memory is the game.** The moment-to-moment sim (walking, working) is scaffolding. The payoff is NPCs referencing specific past events unprompted — that's what has to work first and best.
- **Cheap loop, expensive voice.** Anything that runs every tick (movement, retrieval math) must be free/local. Anything that costs a model call (planning, reflection, dialogue) runs on a budget.

---

2. Tech Stack

Layer	Choice	Notes
Client	Single-file HTML/JS + Canvas (your standard pattern)	Tilemap renderer, virtual joystick, WebAudio SFX

Dev tool	Claude Code	Builds/iterates the whole project, headless test harness
NPC cognition/dialogue	**Fable 5** (via API)	Handles reflection, planning, and in-character dialogue generation — the "voice" layer
Memory retrieval	**OpenAI API (your key)**	`text-embedding-3-small` for embedding memory entries + cosine-similarity retrieval (this is the actual role embeddings played in the original Smallville paper — cheap, fast, doesn't need a reasoning model)
Persistence	localStorage or IndexedDB for solo play; optional lightweight backend later if you want multi-session/server-authoritative sim	Keeps single-file portability for v1
Art	Kenmi Cute Fantasy packs (owned) + Higgsfield for bespoke building/prop fills	No AI-slop rule applies — Higgsfield gen assets get curated/edited, not dumped in raw

Split rationale: Fable 5 = reasoning/personality (expensive, do it sparingly — once per planning tick, once per conversation turn). OpenAI embeddings = cheap infrastructure (every memory write gets embedded, every retrieval does a similarity search). Don't burn Fable 5 calls on retrieval math.

Cost/latency budgeting (concrete): with 8 NPCs, 1 planning call/NPC/day, reflection roughly every 4-6 in-game hours (~3-4/NPC/day), and dialogue only on player-initiated conversation, a full simulated day costs roughly 8 planning + ~30 reflection calls to Fable 5 — cheap enough to run many simulated days during testing. Embedding calls are ~1 per memory write (dozens/day/NPC) but `text-embedding-3-small` is cheap enough to not think about.

---

3. Core Architecture (Generative Agent Loop)

Each NPC runs this loop, staggered on an offset tick (e.g. NPC index mod N) so not

everyone thinks on the same frame and Fable 5 calls don't spike simultaneously:

1. **Perceive** — nearby entities, objects, events within perception radius get logged as raw observations. Perception radius is small (a few tiles) so NPCs don't omnisciently know things happening across town — this is what makes gossip/rumor propagation feel earned rather than telepathic.
2. **Memory Stream** — every observation/action/dialogue is a timestamped memory object with **recency**, **importance** (1-10, scored once at creation — cheap heuristic scoring, not a model call: combat/conflict/gift/confession-type events score high, routine movement scores low), and **relevance** (computed at retrieval time via embedding similarity to current context).
3. **Retrieval** — score = `α·recency + β·importance + γ·relevance`, recency as exponential decay (`0.99^hours_elapsed` is a reasonable starting decay), all three normalized to [0,1] before weighting. Top-N (start with N=10-15) memories pulled for any decision, sorted by score descending.
4. **Reflection** — periodically (every N in-game hours, or when the sum of importance scores since the last reflection crosses a threshold — start at ~150), Fable 5 synthesizes recent memories into a higher-level insight ("I think Mara doesn't trust me since the market incident"). Reflections get stored back into the memory stream as their own memory type, so they can be retrieved later just like observations — this is how opinions compound over time instead of resetting.
5. **Planning** — each in-game morning, Fable 5 generates a rough daily schedule for the NPC given their role, relationships, retrieved high-importance memories, and yesterday's unresolved threads. Plans are coarse (a handful of anchor blocks, not minute-by-minute) and get filled in procedurally by the scheduler/pathfinder.
6. **Reaction/interruption** — before blindly executing the next plan step, check whether a new high-importance perception warrants a re-plan (player approaches, another NPC does something notable, a scheduled town event fires). This is a cheap importance-threshold check, not a Fable 5 call — only escalate to a re-planning call if the interruption clears the threshold.
7. **Action/Dialogue** — moment-to-moment movement follows the current plan step via pathfinding; conversations trigger a Fable 5 call using retrieved memories + relationship state + the last few dialogue turns as context.

This mirrors the original Smallville paper's loop — you're not reinventing it, just re-skinning the model calls to Fable 5 + OpenAI embeddings instead of GPT-3.5 + ada.

---

4. World Design

- **Map**: tilemap grid, districts (residential, market, tavern/social hub, workplaces, outskirts). Start small — 1 town, ~6-10 buildings — prove the loop before scaling.
- **Buildings**: each has an interior state (open/closed by time of day), an owner/staff NPC, and a function tag (`home` , `shop` , `work` , `social`) that feeds into planning (NPCs query "where can I do X" against building function tags rather than Fable 5 inventing a location out of nothing).
- **Pathfinding**: grid-based A* is enough at this scale — no need for a full navmesh. Cache paths between fixed points (home↔work↔social hub) since those are walked constantly.
- **Time system**: single day-cycle clock driving schedules, shop hours, lighting/palette shifts (you already do 5-act palette arcs in Bugler — reuse that instinct for day/night here: dawn/day/dusk/night palette swaps, not just a brightness slider).
- **Town events (optional but cheap to add later)**: a small calendar of scheduled happenings (market day, festival) that everyone's plan has to route around — a good stress test for the interruption/re-plan system once the core loop is solid.

---

5. NPCs

- **Roster**: define a locked cast (suggest starting at 6-8, matching your "locked four-character cast" pattern from Bugler — small casts let relationships get deep instead of shallow).
- **Per-NPC sheet**: name, role/occupation, home + workplace, personality traits (3-5 adjectives, feeds Fable 5 system prompt directly), starting relationships (numeric affinity + a short relationship memory), daily routine skeleton (wake/work/social/sleep anchor points that planning fills in around).
- **Relationships**: stored as a directed graph — NPC A's opinion of NPC B is not necessarily symmetric. Affinity score (-1 to 1) + a rolling "relationship memory summary" updated on reflection, not on every interaction (keeps summaries coherent instead of thrashing).
- **Items/Inventory**: simple — NPCs and buildings can hold items (tools, gifts, shop goods). Items are referenceable in memory ("gave Tomas the bread") so they matter for relationship/plan context, not just simulation for its own sake.
- **Player relationship**: treat the player as a special-cased "NPC" in the relationship graph so the same affinity/memory-summary machinery applies to how NPCs feel about the

player, instead of a bolted-on separate reputation system.

---

6. Data Schemas

```
// Agent
{
  "id": "npc_mara",
  "name": "Mara",
  "role": "baker",
  "traits": ["warm", "gossipy", "proud"],
  "home": "building_bakery_house",
  "workplace": "building_bakery",
  "schedule_anchors": [["06:00","wake"], ["07:00","open_shop"], ["19:00","close_s
  "relationships": {
    "npc_tomas": { "affinity": 0.6, "summary": "Trusts him since he fixed her ove
    "player": { "affinity": 0.1, "summary": "Hasn't formed a strong opinion yet."
  },
  "inventory": ["bread_loaf_x4", "spare_key"],
  "current_plan": ["open_shop@07:00", "market_gossip@11:00", "close_shop@19:00",
  "last_reflection_tick": "day3_10:00"
}


// Memory object
{
  "id": "mem_00234",
  "npc": "npc_mara",
  "timestamp": "day3_14:32",
  "type": "observation | reflection | plan | dialogue",
  "text": "Saw the player help Tomas carry crates.",
  "importance": 6,
  "embedding": [ /* vector, OpenAI text-embedding-3-small */ ],
  "related_ids": ["mem_00219"]
}


// Building
{
  "id": "building_bakery",
  "name": "Mara's Bakery",
  "function": "shop",
  "owner": "npc_mara",
  "open_hours": ["07:00", "19:00"],
  "interior_tiles": "bakery_interior_01",
```

```
  "items_available": ["bread_loaf"]
}


// Relationship edge (if you want it queryable independent of the agent object, f
{
  "from": "npc_mara",
  "to": "player",
  "affinity": 0.1,
  "summary": "Hasn't formed a strong opinion yet.",
  "last_updated": "day3_10:00"
}
```

---

7. API Integration Plan

- **Fable 5 calls** (expensive, rationed): daily planning (1/NPC/day), reflection (triggered, not constant), in-conversation dialogue turns (1/turn), escalated re-plans on high-importance interruptions.
- **OpenAI calls** (cheap, constant): embed every new memory on write; embed the current query context at retrieval time; cosine similarity done client-side in JS — no need to round-trip a vector DB for a town this size (dozens–low hundreds of memories per NPC is trivial to brute-force score).
- **Prompt construction discipline**: every Fable 5 call should get (a) the NPC's static sheet (traits, role, relationships summary), (b) the top-N retrieved memories relevant to the current context, (c) a tight instruction for the expected output format (a plan array, a single reflection sentence, a dialogue line) — keep these prompts terse and structured so Claude Code can unit-test the parsing without the model rambling into freeform prose.
- **Key handling**: store both API keys in a local config (never hardcoded/committed); Claude Code should scaffold a settings panel or local config file for key entry rather than baking keys into the shipped HTML. If you ever want to share this build, the key-entry screen also doubles as the thing that stops your credits being someone else's free NPC brain.

---

8. Testing Strategy (matches your headless-harness pattern)

- **Retrieval math**: unit-test the scoring function against a hand-written memory log with known expected top-N results — validate before any live model call is involved.

- **Simulated days**: run N in-game days headless (no rendering, no player) with mocked or throttled model calls, and assert nothing breaks — no NPC stuck off-schedule, no relationship affinity drifting to NaN, no orphaned memory references.
- **Dialogue regression**: snapshot a handful of scripted conversation contexts and check Fable 5 responses stay in-character and correctly reference the intended memories, not just "sounds plausible."
- **Cost ceiling test**: headless-run a week of simulated days and log total model calls, to catch any accidental per-tick call instead of per-planning-cycle call before it burns credits.

---

9. Build Phases

1. **Skeleton** — tilemap render, player movement/joystick, one building, one NPC walking a hardcoded loop. No LLM yet.
2. **Memory + retrieval** — wire OpenAI embeddings, get retrieval scoring working against a hand-written memory log (no live LLM calls yet — validate the math per the testing strategy above).
3. **Planning + dialogue** — hook in Fable 5 for daily plans and conversation. Single NPC, prove the full loop end to end, including interruption/re-plan handling.
4. **Scale cast** — expand to full roster, wire relationships graph, add reflection cadence.
5. **World fill** — remaining buildings, items, shop hours, day/night palette, pathfinding polish.
6. **Polish/juice** — dialogue UI, ambient SFX, headless test harness running full simulated weeks, cost ceiling checks before final art pass.

---

10. Open Decisions (flag if you want to change defaults)

- Cast size for v1 (default: 6-8, locked cast).
- Persistence: localStorage-only vs. lightweight backend.
- Whether OpenAI key is *only* for embeddings, or you also want it as a fallback/cheaper LLM for low-stakes NPC chatter to save Fable 5 credits.
- Whether town events (festivals/market days) are in scope for v1 or deferred to a post-

launch pass.

Proceeding on defaults above unless you want to change any of them — happy to start on Phase 1 skeleton in Claude Code whenever you're ready.